Niveau : S5 LICENCE GÉNIE LOGICIEL

Rapport de Système de Gestion des Livraisons-QuickDoor

Module : Programmation Framework SpringBoot

Réalisé par :

Mr. Omar SAFARI.

Encadré à FSK par :

Dr. Khalid HOUSNI

Année Universitaire 2025 – 2026

Table des matières

Liste des Figures III
Liste des Abréviations IV
Introduction 5
2 Présentation du projet 6
2.1 Contexte du projet 6
2.2 Objectifs du projet 6
2.3 Description de l'application 6
2.4 Fonctionnalités principales 7
2.5 Architecture générale 8
3 Analyse des besoins 8
3.1 Besoins fonctionnels 9
3.2 Besoins non fonctionnels 9
4 Conception UML 9
4.1 Diagramme de cas d'utilisation 10
4.2 Diagramme de classes 11
4.3 Diagramme de séquence 11
5 Technologies utilisées 13
5.1 Backend 13
5.2 Frontend 13
5.3 Base de données 13
5.4 Outils de développement 14
6 Réalisation (Implémentation) 14
6.1 Structure du projet 14
6.2 Base de données 15
6.2.1 Modèles principaux. 15
6.2.2 Exemple de modèle Django 16
6.2.3 Relations entre les modèles 17
6.3 Implémentation des fonctionnalités principales 17
6.3.1 Gestion des produits 17
6.3.2 Gestion du panier 18
6.3.3 Gestion des commandes 18
6.3.4 Authentification des utilisateurs 18
7 Interface utilisateur 19
7.1 Page d'accueil 20
7.2 Page des produits 20
7.3 Page du panier 21
7.4 Page des commandes 22
8 Conclusion et perspectives 23

Liste des Figures

Figure 2 : Interface principale de l'application QuickDoor 7
Figure 3 : Architecture MVC de l'application QuickDoor 8
Figure 4 : Diagramme de classes de l'application QuickDoor..... 11
Figure 5 : Diagramme de séquence du processus de commande dans l'application QuickDoor..... 12
Figure 6 : Structure du projet QuickDoor sous Django..... 15
Figure 7 : Interface Products 18
Figure 8 :Interface Cart 19
Figure 9 : Interface Orders 19
Figure 10 : Interface de la page d'accueil de l'application QuickDoor 20
Figure 11 : Interface de la page des produits..... 21
Figure 12 : Interface de la page du panier 22
Figure 13 : Interface de la page des commandes..... 22

Liste des Abréviations

HTML	HyperText Markup Language
CSS	Cascading Style Sheets
VS Code	Visual Studio Code
URL	Uniform Resource Locator
IDE	Integrated Development Environment
UX	User Experience
UI	User Interface
GSAP	GreenSock Animation Platform

Introduction

Dans le cadre de notre formation en développement web, nous avons réalisé un projet intitulé **“Système de Gestion des Livraisons”**.
Ce projet a pour objectif de concevoir et développer une application web permettant de gérer efficacement les opérations de livraison entre les clients et les livreurs.

Avec l’augmentation des services de livraison (e-commerce, restauration, colis…), il devient essentiel de disposer d’un système informatisé capable de :

- gérer les utilisateurs (clients, livreurs, administrateurs),
- organiser les commandes de livraison,
- suivre l’état des livraisons en temps réel.

L’application développée permet ainsi aux clients de créer des demandes de livraison, de suivre leur statut (en préparation, en transit, livrée), et aux administrateurs d’attribuer les livraisons aux livreurs disponibles.

Ce projet repose sur une architecture moderne basée sur :

- un **backend en Spring Boot (Java)** pour la gestion des données et des APIs,
- un **frontend en React.js** pour l’interface utilisateur,
- une **base de données MySQL** pour le stockage des informations.

L’objectif principal est de proposer une solution simple, efficace et responsive, adaptée aussi bien aux ordinateurs qu’aux appareils mobiles.

2 Présentation du projet

L'objectif principal de ce portfolio est de **mettre en valeur mes compétences en développement web**, tout en offrant une **navigation claire et intuitive** à l'utilisateur. Grâce à un design responsive, le site s'adapte aussi bien aux écrans d'ordinateur qu'aux tablettes et smartphones.

Le site contient plusieurs sections distinctes :

- Une page d'accueil dynamique avec une animation d'introduction,
- Une section "À propos" pour me présenter de manière professionnelle,
- Une section "Compétences" illustrée par des icônes et effets visuels,
- Une galerie de projets réalisés,
- Un aperçu des services que je peux offrir,
- Un formulaire de contact stylisé.

Ce projet m'a permis d'appliquer mes connaissances en front-end et d'améliorer mes compétences en design web, tout en mettant en avant ma propre identité visuelle.

2.1 Contexte du projet

Avec l'augmentation de la demande des services de livraison, il devient nécessaire de proposer des solutions digitales permettant de simplifier le processus de commande et de suivi des livraisons.

Le projet **QuickDoor** s'inscrit dans cette optique en proposant une application web moderne permettant de connecter les utilisateurs aux services de livraison de manière rapide et efficace.

2.2 Objectifs du projet

L'objectif principal de ce projet est de concevoir et développer une application web de livraison offrant une expérience utilisateur simple, rapide et efficace.

Ce projet vise à répondre aux besoins des utilisateurs en facilitant le processus de commande et en améliorant le suivi des livraisons.

Les objectifs spécifiques du projet sont :

- Faciliter la consultation des produits disponibles
- Permettre aux utilisateurs de passer des commandes en ligne
- Mettre en place un système de gestion de panier
- Assurer le suivi des livraisons en temps réel
- Offrir une interface d'administration pour la gestion des données

2.3 Description de l'application

QuickDoor est une application web développée à l'aide du framework Django, permettant de gérer un système de livraison en ligne de manière efficace.

L'application offre aux utilisateurs la possibilité de parcourir une liste de produits, de les ajouter à un panier, puis de valider leurs commandes en quelques étapes simples.

Une fois la commande effectuée, l'utilisateur peut suivre son état (en attente, en cours de livraison, livrée) grâce à un système de suivi intégré.

De plus, l'application inclut un système de notifications permettant d'informer l'utilisateur des mises à jour liées à ses commandes.

Une interface d'administration est également disponible pour gérer les produits, les commandes et les utilisateurs, facilitant ainsi la gestion globale du système.

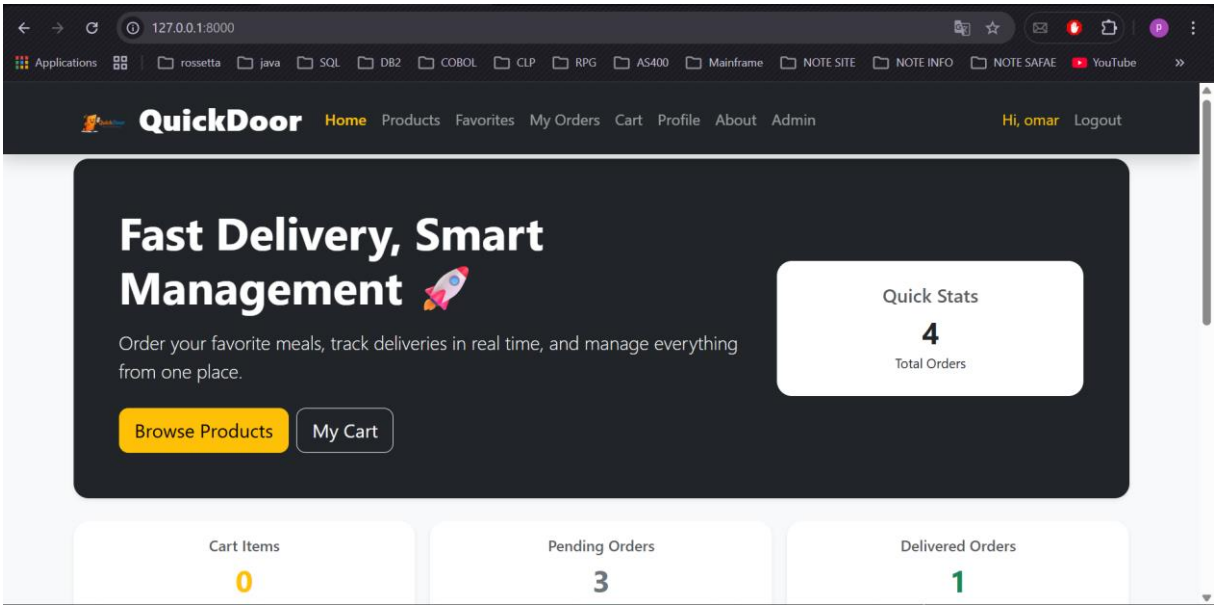


Figure 1 : Interface principale de l'application QuickDoor

2.4 Fonctionnalités principales

L'application **QuickDoor** propose un ensemble de fonctionnalités permettant de répondre aux besoins des utilisateurs et de faciliter la gestion des services de livraison.

Les principales fonctionnalités sont les suivantes :

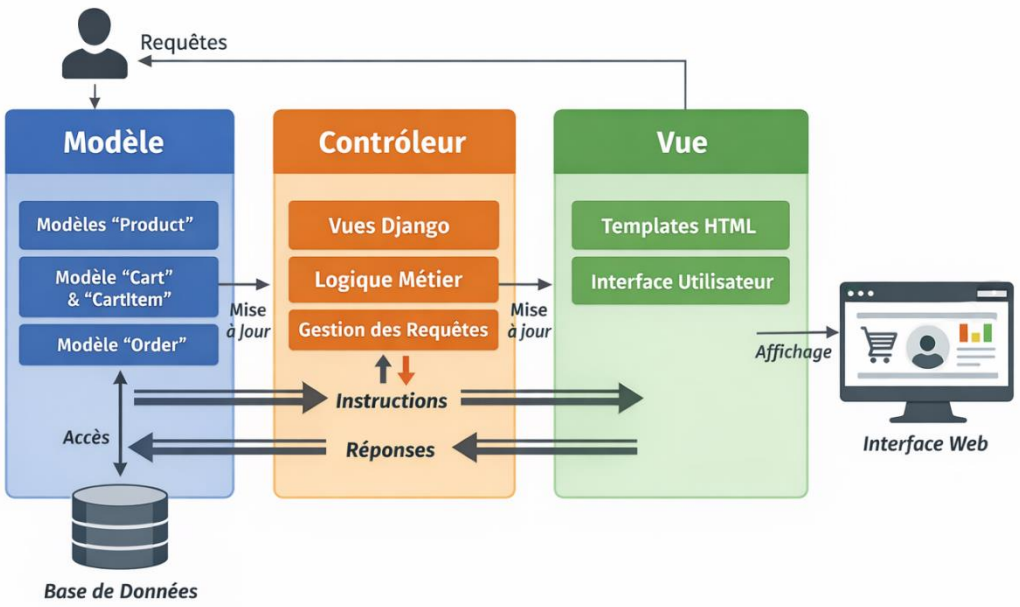
- **Gestion des utilisateurs** : inscription, connexion et gestion du profil
- **Consultation des produits** : affichage des produits disponibles
- **Gestion du panier** : ajout, suppression et modification des articles
- **Passage de commande** : validation et enregistrement des commandes
- **Suivi des livraisons** : visualisation de l'état de la commande
- **Notifications** : alertes sur les mises à jour des commandes
- **Interface administrateur** : gestion des produits, commandes et utilisateurs

2.5 Architecture générale

Le projet **QuickDoor** est basé sur l'architecture **MVC (Model – View – Template)** propre au framework Django.

Cette architecture permet de séparer les différentes parties de l'application afin de faciliter la maintenance et l'évolution du système.

- **Model** : représente les données de l'application (produits, commandes, utilisateurs) et gère les interactions avec la base de données.
- **View** : contient la logique métier de l'application et traite les requêtes des utilisateurs.
- **Template** : correspond à l'interface utilisateur affichée dans le navigateur.



Architecture MVC de l'application QuickDoor

Figure 2 : Architecture MVC de l'application QuickDoor

3 Analyse des besoins

L'analyse des besoins constitue une étape essentielle dans le développement de toute application informatique. Elle permet d'identifier les attentes des utilisateurs et de définir les fonctionnalités nécessaires au bon fonctionnement du système.

Dans le cadre du projet **QuickDoor**, cette phase a permis de déterminer les besoins fonctionnels et non fonctionnels afin de concevoir une application adaptée aux exigences des utilisateurs.

3.1 Besoins fonctionnels

Les besoins fonctionnels représentent les fonctionnalités que doit offrir l'application afin de répondre aux attentes des utilisateurs.

Dans le cadre de l'application **QuickDoor**, les besoins fonctionnels sont définis comme suit :

- **Inscription et authentification** : permettre aux utilisateurs de créer un compte et de se connecter
- **Consultation des produits** : afficher la liste des produits disponibles
- **Gestion du panier** : ajouter, supprimer ou modifier les produits dans le panier
- **Passage de commande** : valider et enregistrer une commande
- **Suivi des commandes** : permettre à l'utilisateur de suivre l'état de sa livraison
- **Gestion des notifications** : informer l'utilisateur des mises à jour de sa commande
- **Administration** : gérer les produits, les commandes et les utilisateurs via une interface admin

3.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les exigences liées à la qualité du système, telles que la performance, la sécurité et l'ergonomie.

Pour l'application **QuickDoor**, les besoins non fonctionnels sont les suivants :

- **Performance** : l'application doit être rapide et répondre efficacement aux requêtes des utilisateurs
- **Sécurité** : assurer la protection des données des utilisateurs (authentification, gestion des accès)
- **Ergonomie** : proposer une interface simple, intuitive et facile à utiliser
- **Disponibilité** : garantir un accès continu à l'application
- **Maintenabilité** : faciliter la mise à jour et l'évolution du système

4 Conception UML

La conception UML constitue une étape importante dans le développement d'une application informatique. Elle permet de représenter de manière claire et structurée le fonctionnement du système ainsi que les interactions entre ses différents composants.

Dans le cadre du projet **QuickDoor**, la modélisation UML a été utilisée afin de mieux comprendre les besoins du système et de faciliter sa réalisation.

Cette partie présente les principaux diagrammes réalisés pour l'application, notamment le diagramme de cas d'utilisation, le diagramme de classes et le diagramme de séquence.

4.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet de représenter les interactions entre les utilisateurs (acteurs) et le système.

Dans le cas de l'application **QuickDoor**, ce diagramme illustre les principales fonctionnalités offertes aux utilisateurs ainsi que les actions qu'ils peuvent effectuer.

Les principaux acteurs du système sont :

- **Utilisateur** : client qui utilise l'application
- **Administrateur** : responsable de la gestion des données

Les cas d'utilisation principaux sont :

- S'inscrire / Se connecter
- Consulter les produits
- Ajouter au panier
- Passer commande
- Suivre la livraison
- Gérer les produits (admin)
- Gérer les commandes (admin)

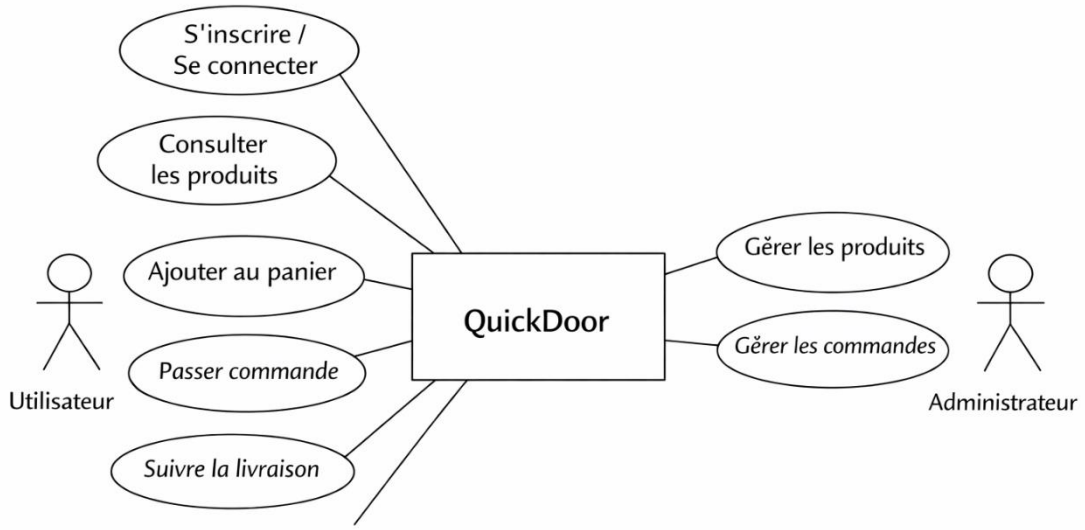


Figure 3 : Diagramme de cas d'utilisation de l'application QuickDoor

4.2 Diagramme de classes

Le diagramme de classes permet de représenter la structure statique de l'application en identifiant les principales entités du système, leurs attributs et les relations entre elles.

Dans le projet **QuickDoor**, ce diagramme met en évidence les classes principales telles que l'utilisateur, le produit, la commande, le panier et la notification.

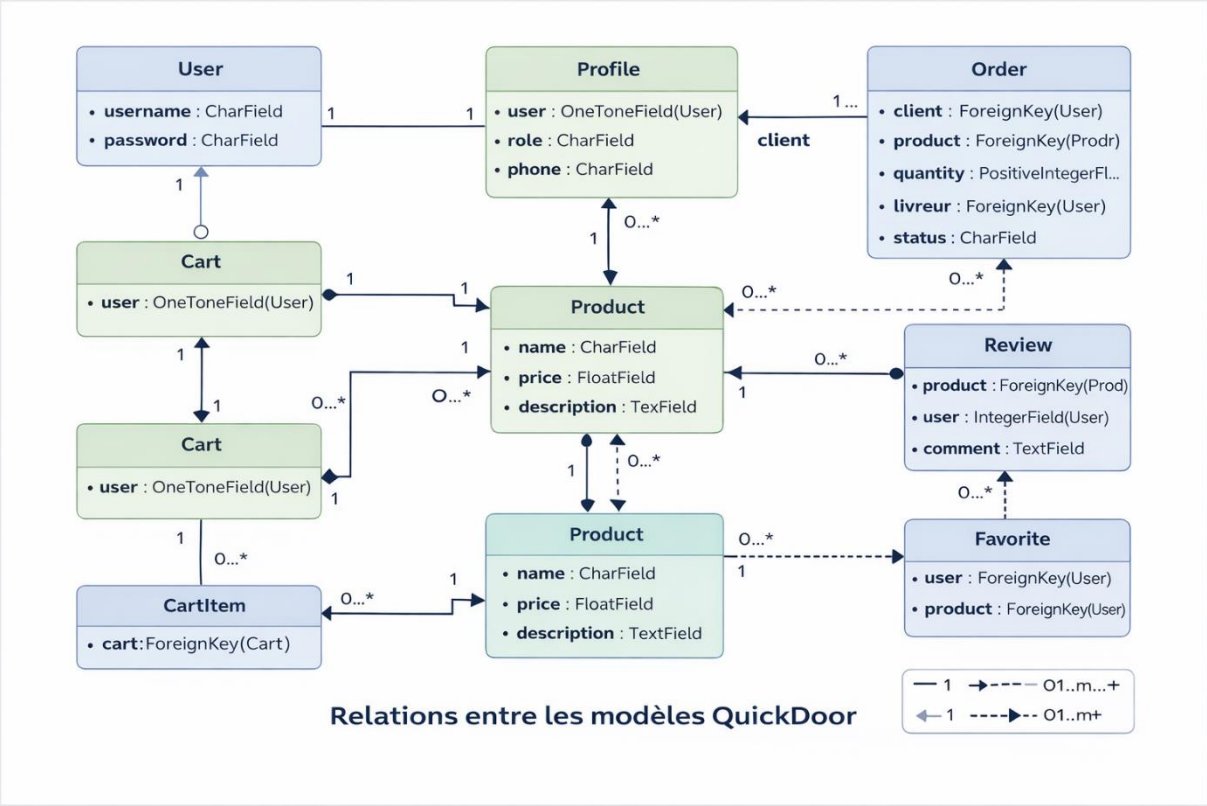


Figure 3 : Diagramme de classes de l'application QuickDoor

Le diagramme de classes présente les différentes entités du système ainsi que leurs relations.

Dans l'application **QuickDoor**, les principales classes sont :

- **Utilisateur** : représente les clients de l'application
- **Produit** : contient les informations des produits disponibles
- **Commande** : représente les commandes effectuées par les utilisateurs
- **Panier** : permet de gérer les produits sélectionnés avant la commande
- **Notification** : informe l'utilisateur des mises à jour liées à ses commandes

4.3 Diagramme de séquence

Le diagramme de séquence permet de représenter l'enchaînement chronologique des interactions entre les différents éléments du système lors de l'exécution d'un scénario précis.

Dans le cas de l'application **QuickDoor**, ce diagramme illustre les étapes suivies lorsqu'un utilisateur sélectionne des produits, les ajoute au panier, valide sa commande, puis reçoit une confirmation.

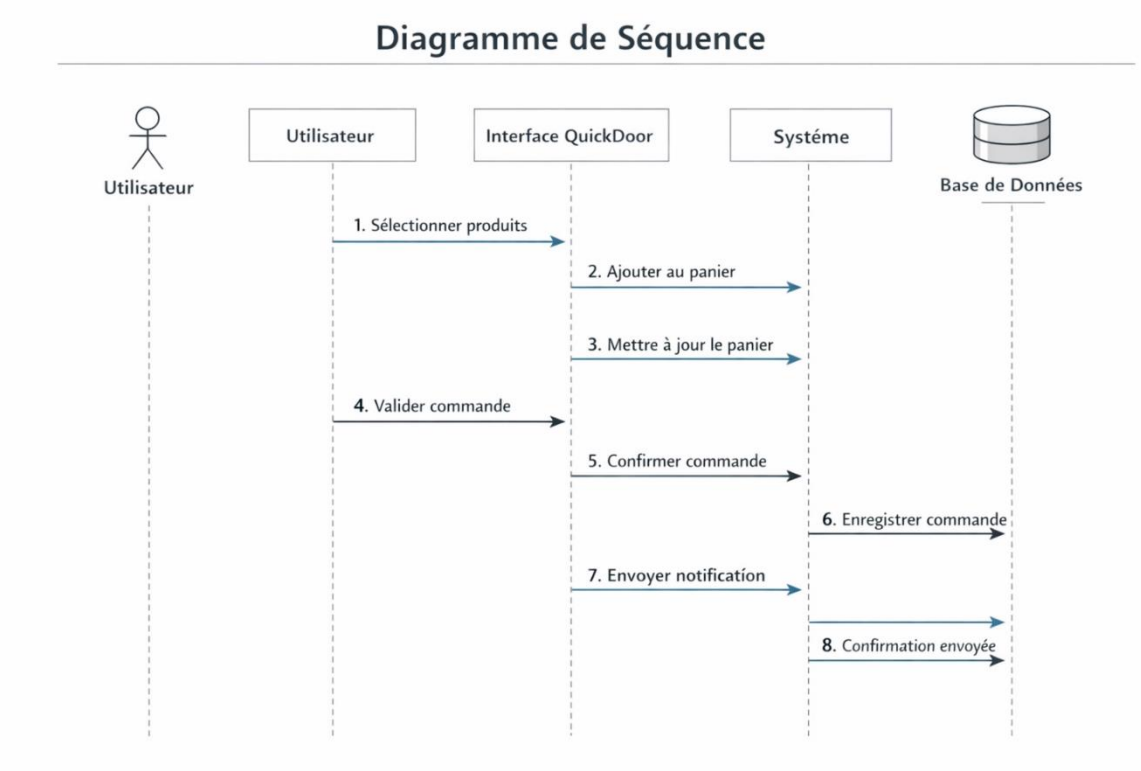


Figure 4 : Diagramme de séquence du processus de commande dans l'application QuickDoor

Ce diagramme montre les différentes étapes du processus de commande dans l'application **QuickDoor**.

L'utilisateur commence par sélectionner les produits, puis les ajoute à son panier. Ensuite, il valide sa commande via l'interface de l'application.

Le système traite la demande, enregistre la commande dans la base de données et envoie une notification de confirmation à l'utilisateur.

Ce processus garantit une interaction fluide entre les différents composants du système et permet d'assurer le bon déroulement de la commande.

5 Technologies utilisées

Le développement de l'application **QuickDoor** a nécessité l'utilisation de plusieurs technologies permettant d'assurer la performance, la sécurité et une bonne expérience utilisateur.

Ces technologies couvrent à la fois le développement backend, frontend, ainsi que la gestion de la base de données.

5.1 Backend

Le backend de l'application **QuickDoor** a été développé en utilisant le framework **Django**, basé sur le langage **Python**.

Django est un framework web puissant qui permet de créer des applications sécurisées, rapides et bien structurées. Il repose sur une architecture **MVC (Model – View – Template)**, facilitant la séparation des différentes parties du système.

Grâce à Django, nous avons pu gérer les modèles de données, la logique métier ainsi que les interactions avec la base de données de manière efficace.

Le langage **Python** a été utilisé pour :

- Développer la logique de l'application
- Gérer les requêtes et les traitements
- Assurer la communication avec la base de données

5.2 Frontend

Le frontend de l'application **QuickDoor** a été réalisé en utilisant les technologies **HTML**, **CSS** et **Bootstrap**.

Ces technologies permettent de créer des interfaces utilisateur modernes, responsives et faciles à utiliser.

- **HTML** : utilisé pour structurer les صفحات web
- **CSS** : utilisé pour le style et la mise en forme
- **Bootstrap** : utilisé pour améliorer le design et rendre l'interface responsive

5.3 Base de données

La gestion des données dans l'application **QuickDoor** est assurée par le système de base de données **SQLite**, intégré par défaut avec le framework Django.

SQLite est une base de données légère, simple à utiliser et adaptée aux applications de petite et moyenne taille.

Elle permet de stocker :

- Les informations des utilisateurs
- Les produits
- Les commandes
- Les paniers
- Les notifications

5.4 Outils de développement

Le développement de l'application **QuickDoor** a été réalisé en utilisant plusieurs outils facilitant la programmation, le test et la gestion du projet.

- **Visual Studio Code** : éditeur de code utilisé pour développer l'application
- **Git** : utilisé pour la gestion des versions du projet
- **Navigateur web** : utilisé pour tester et visualiser l'application

6 Réalisation (Implémentation)

Cette partie présente les différentes étapes de la réalisation de l'application **QuickDoor**, depuis la mise en place de l'environnement de développement jusqu'à l'implémentation des fonctionnalités principales.

Elle permet de décrire la structure du projet, l'organisation du code ainsi que les choix techniques adoptés lors du développement.

6.1 Structure du projet

Le projet **QuickDoor** est structuré selon l'architecture du framework Django, en séparant les différentes fonctionnalités en عدة applications (apps).

Cette organisation permet de faciliter la maintenance, la lisibilité du code et l'évolution du projet.

Le projet contient plusieurs composants principaux :

- **config** : contient les fichiers de configuration du projet
- **orders** : gère les commandes des utilisateurs
- **tracking** : permet de suivre l'état des livraisons
- **notifications** : gère les notifications envoyées aux utilisateurs
- **templates** : contient les interfaces utilisateur (HTML)
- **static** : contient les fichiers statiques (CSS, images, logo…)

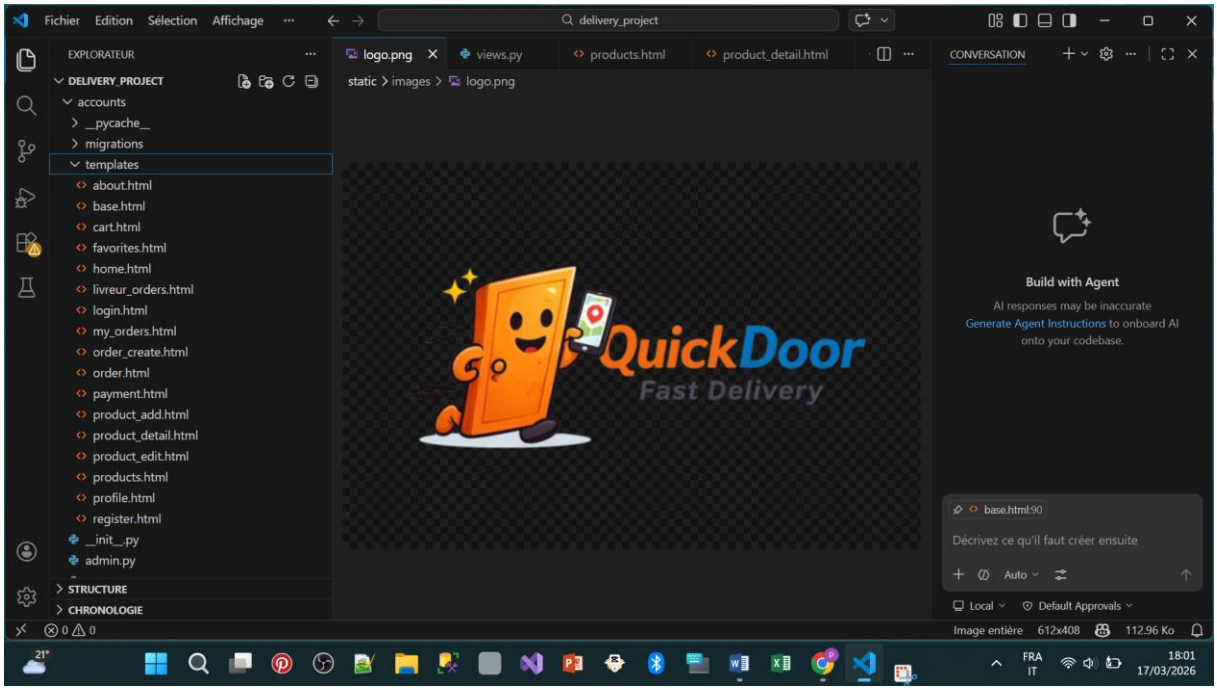


Figure 5 : Structure du projet QuickDoor sous Django

La structure du projet est organisée en plusieurs dossiers et fichiers permettant de séparer les composants de l'application.

On remarque la présence de plusieurs templates HTML pour gérer les différentes pages comme l'accueil, le panier, les commandes et le profil utilisateur.

Cette organisation permet une meilleure lisibilité du code et facilite le développement et la maintenance de l'application.

6.2 Base de données

La base de données constitue un élément essentiel dans le fonctionnement de l'application **QuickDoor**, car elle permet de stocker et d'organiser l'ensemble des informations manipulées par le système.

Dans ce projet, la base de données utilisée est **SQLite**, intégrée par défaut avec le framework Django. La gestion des données se fait à travers l'ORM de Django, qui permet de définir les tables sous forme de classes Python appelées **modèles**.

Les modèles implémentés dans l'application **QuickDoor** représentent les différentes entités du système, notamment les profils utilisateurs, les produits, les commandes, les paniers, les avis et les favoris.

6.2.1 Modèles principaux.

Les principaux modèles de la base de données sont les suivants :

- **Profile** : permet de compléter les informations de l'utilisateur, notamment son rôle dans le système (client, livreur ou administrateur) ainsi que son numéro de téléphone.
- **Product** : représente les produits disponibles dans l'application avec leurs informations principales, telles que le nom, la description, le prix, l'image et la catégorie.
- **Order** : représente les commandes passées par les clients, avec le produit commandé, la quantité, le livreur affecté et le statut de la commande.
- **Cart** : représente le panier de l'utilisateur.
- **CartItem** : représente les éléments contenus dans le panier, avec le produit et la quantité sélectionnée.
- **Review** : permet aux utilisateurs de laisser un avis et une note sur les produits.
- **Favorite** : permet aux utilisateurs d'ajouter des produits à leur liste de favoris.

6.2.2 Exemple de modèle Django

```
class Product(models.Model):

    CATEGORY_CHOICES = [

        ("FOOD", "Food"),

        ("DRINK", "Drink"),

        ("OTHER", "Other"),

    ]

    name = models.CharField(max_length=200)

    description = models.TextField()

    price = models.FloatField()

    image  =  models.ImageField(upload_to="products/",
blank=True, null=True)

    category = models.CharField(

        max_length=20,

        choices=CATEGORY_CHOICES,

        default="OTHER"

    )
```

```
    created_at =
models.DateTimeField(auto_now_add=True)

    def __str__(self):

        return self.name
```

Ce modèle représente l'entité **Product** dans l'application. Il permet de stocker les informations liées aux produits proposés aux utilisateurs, telles que le nom, la description, le prix, l'image, la catégorie et la date d'ajout.

L'utilisation des modèles Django facilite la structuration des données et leur manipulation à travers l'ORM, sans avoir à écrire directement des requêtes SQL complexes.

6.2.3 Relations entre les modèles

Les relations entre les modèles permettent d'assurer la cohérence des données dans l'application. Par exemple :

- Un **utilisateur** possède un seul **profil** grâce à une relation *OneToOne*.
- Un **utilisateur** peut avoir plusieurs **commandes**.
- Un **panier** appartient à un seul utilisateur, mais peut contenir plusieurs **articles** (*CartItem*).
- Un **produit** peut apparaître dans plusieurs commandes, paniers, avis et favoris.
- Un **livreur** peut être associé à plusieurs commandes pour assurer la livraison.

6.3 Implémentation des fonctionnalités principales

L'implémentation des fonctionnalités principales de l'application **QuickDoor** repose sur l'utilisation des vues Django, des templates HTML et des modèles définis précédemment.

Cette partie permet de présenter les principales fonctionnalités développées ainsi que leur fonctionnement au sein de l'application.

Les principales fonctionnalités implémentées sont les suivantes :

6.3.1 Gestion des produits

Cette fonctionnalité permet aux utilisateurs de consulter les produits disponibles dans l'application.

Les produits sont affichés à partir de la base de données en utilisant les vues Django et sont présentés dans des pages HTML dynamiques.

6.3.2 Gestion du panier

L'utilisateur peut ajouter des produits à son panier, modifier les quantités ou supprimer des articles.

Le panier est géré à l'aide des modèles **Cart** et **CartItem**, permettant de stocker temporairement les choix de l'utilisateur avant la validation de la commande.

6.3.3 Gestion des commandes

Une fois le panier validé, une commande est créée dans la base de données.

Le système enregistre les informations المتعلقة بالمنتج، الكمية، المستخدم، وحالة الطلب.

Le suivi de la commande est assuré grâce au champ **status** qui permet de connaître son état (en attente, en cours, livrée…).

6.3.4 Authentification des utilisateurs

L'application intègre un système d'authentification permettant aux utilisateurs de s'inscrire, se connecter et accéder à leurs informations personnelles.

Cette fonctionnalité est basée sur le système d'authentification intégré de Django.

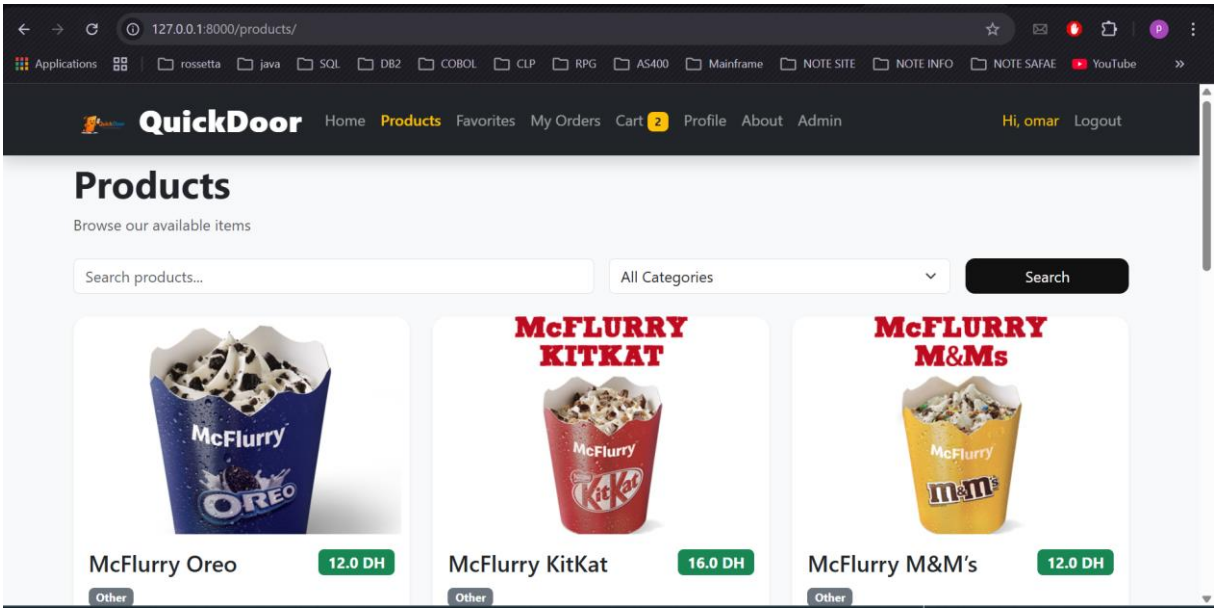


Figure 6 : Interface Products

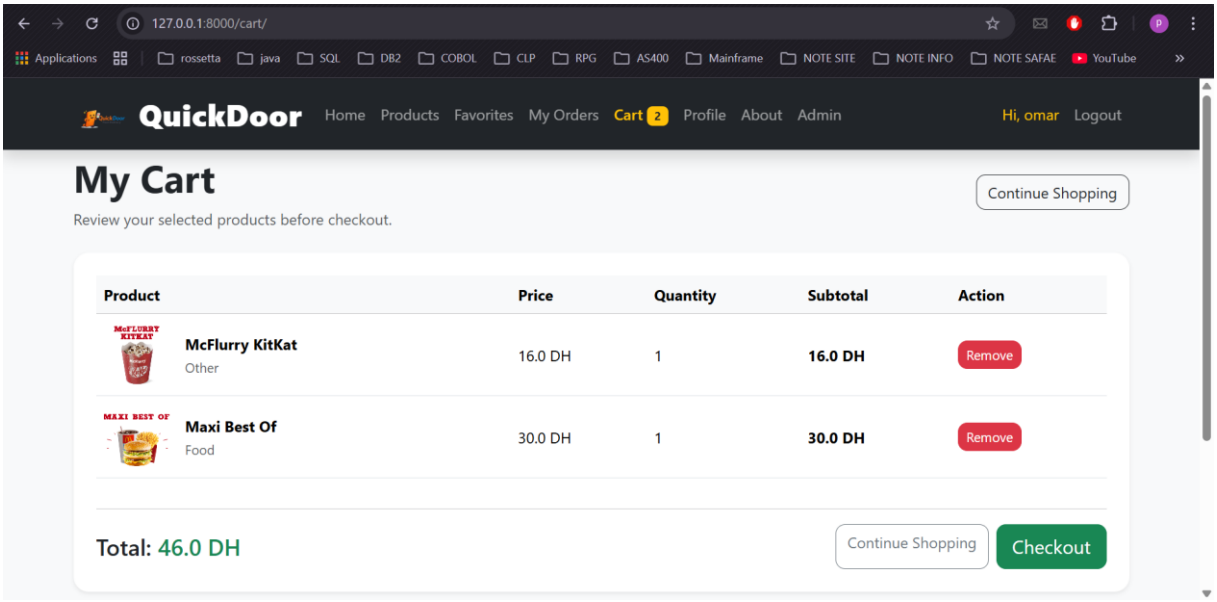


Figure 7 :Interface Cart

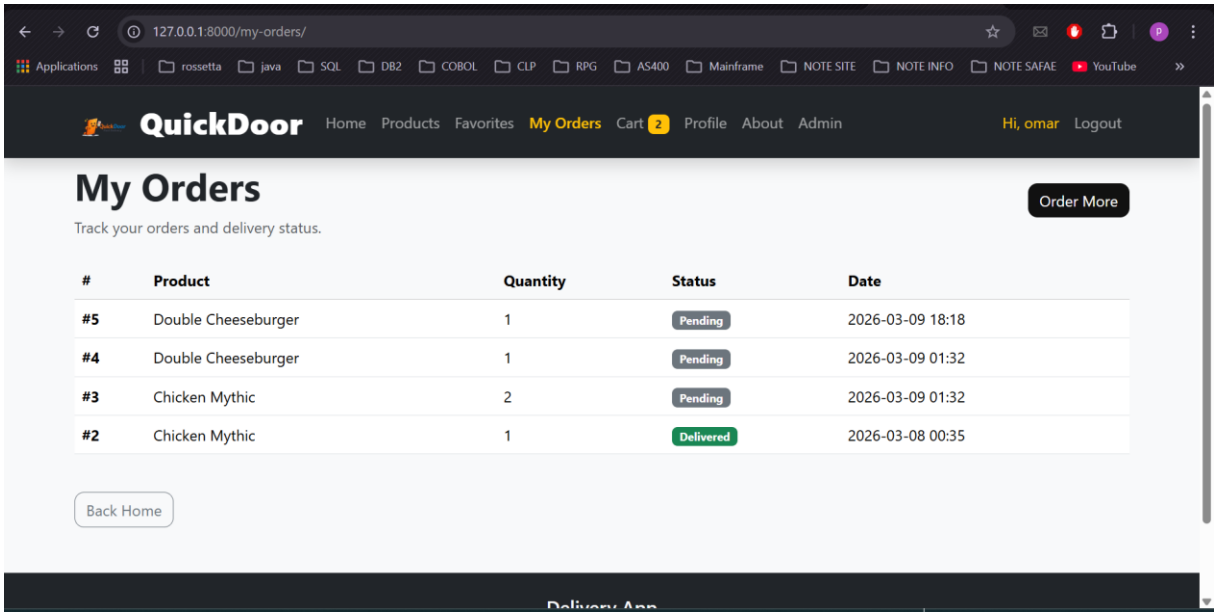


Figure 8 : Interface Orders

7 Interface utilisateur

L'interface utilisateur joue un rôle important dans la réussite d'une application web.

Dans l'application **QuickDoor**, une attention particulière a été portée à la simplicité, à la clarté et à l'ergonomie afin d'offrir une expérience utilisateur fluide et agréable.

7.1 Page d'accueil

La page d'accueil constitue la première interface visible par l'utilisateur après la connexion.

Elle présente une vue générale de l'application, incluant les principales fonctionnalités telles que la navigation vers les produits, le panier et les commandes.

Cette page est conçue de manière claire et intuitive afin de faciliter l'accès rapide aux différentes sections de l'application.

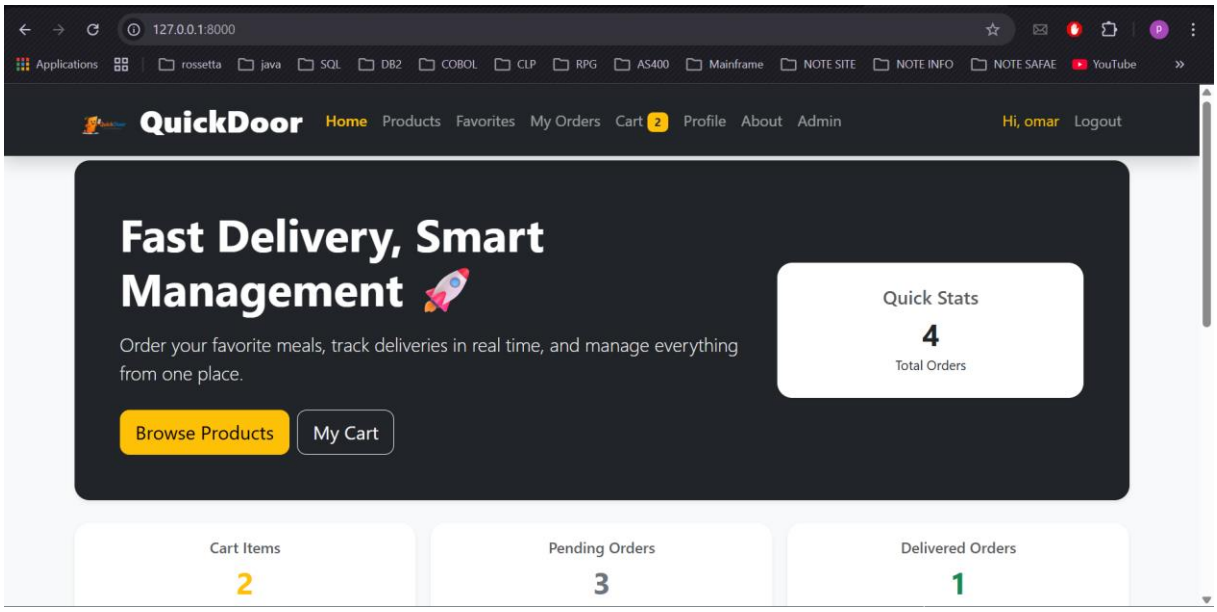


Figure 9 : Interface de la page d'accueil de l'application QuickDoor

On y trouve également des statistiques rapides ainsi que des boutons d'accès direct aux fonctionnalités principales.

7.2 Page des produits

La page des produits permet à l'utilisateur de consulter l'ensemble des produits disponibles dans l'application.

Chaque produit est affiché avec ses informations principales, notamment le nom, le prix, la description et éventuellement une image.

Cette page permet également à l'utilisateur d'ajouter des produits à son panier en quelques clics.

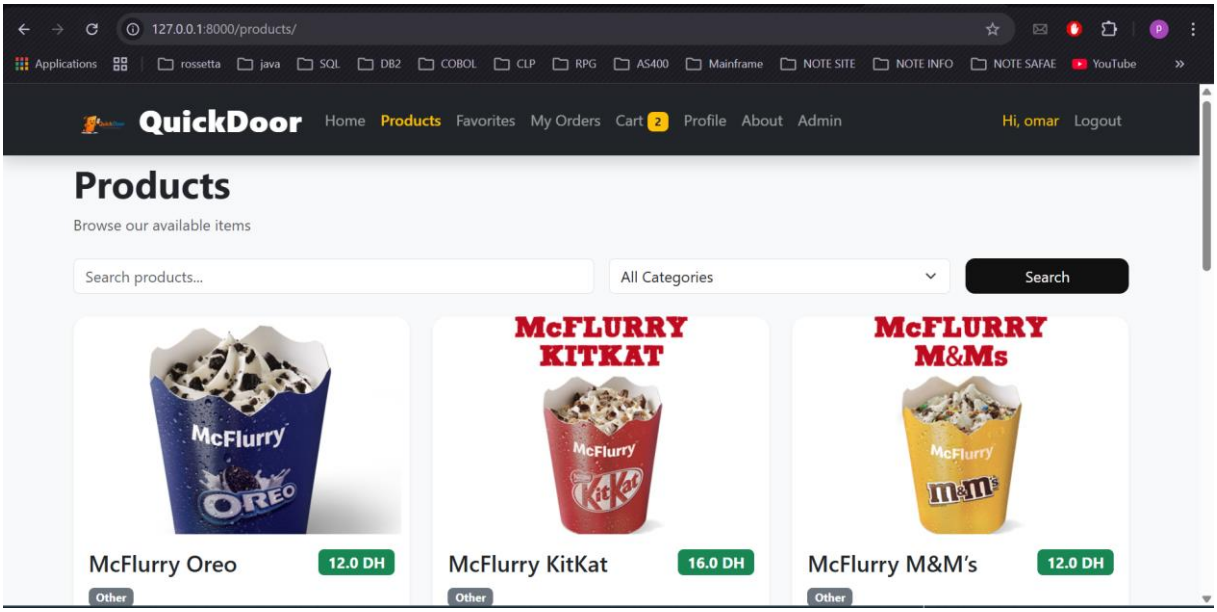


Figure 10 : Interface de la page des produits

Cette interface est conçue pour être simple et intuitive, facilitant la navigation et l'expérience utilisateur.

7.3 Page du panier

La page du panier permet à l'utilisateur de visualiser les produits sélectionnés avant de confirmer sa commande.

Elle affiche les articles ajoutés, leurs quantités ainsi que le montant total à payer.

Cette interface permet également de modifier le contenu du panier ou de supprimer des produits avant la validation finale.

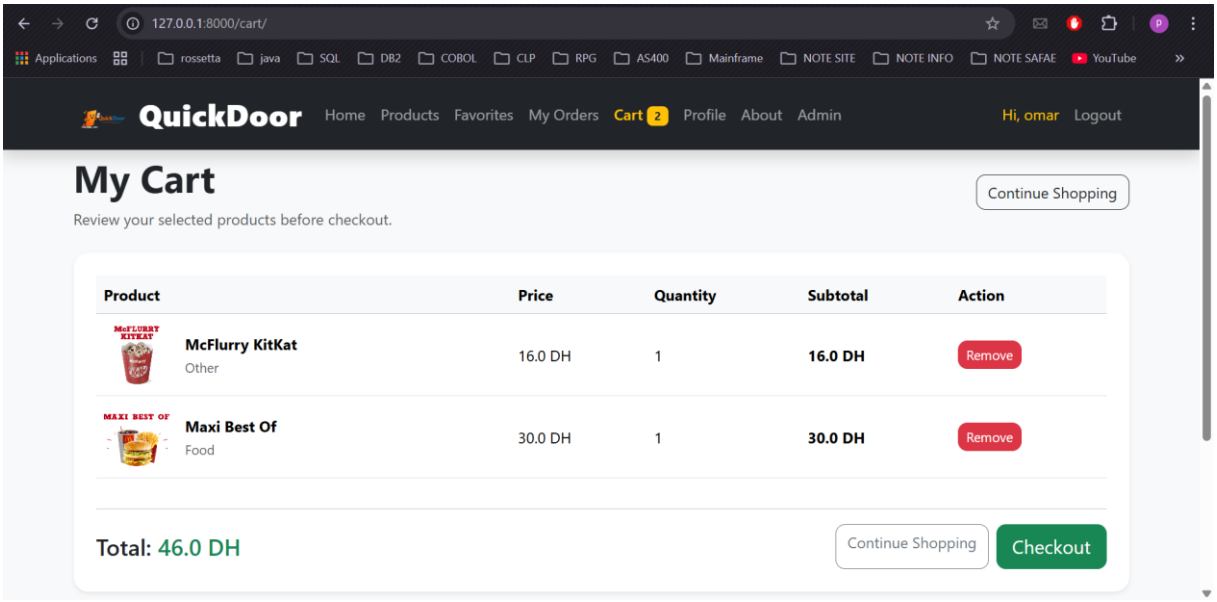


Figure 11 : Interface de la page du panier

7.4 Page des commandes

La page des commandes permet à l'utilisateur de consulter l'historique de ses commandes ainsi que leur état.

Chaque commande affiche les informations principales telles que le produit commandé, la quantité, la date et le statut de la commande (en attente, en cours, livrée…).

Cette fonctionnalité permet à l'utilisateur de suivre facilement ses commandes.

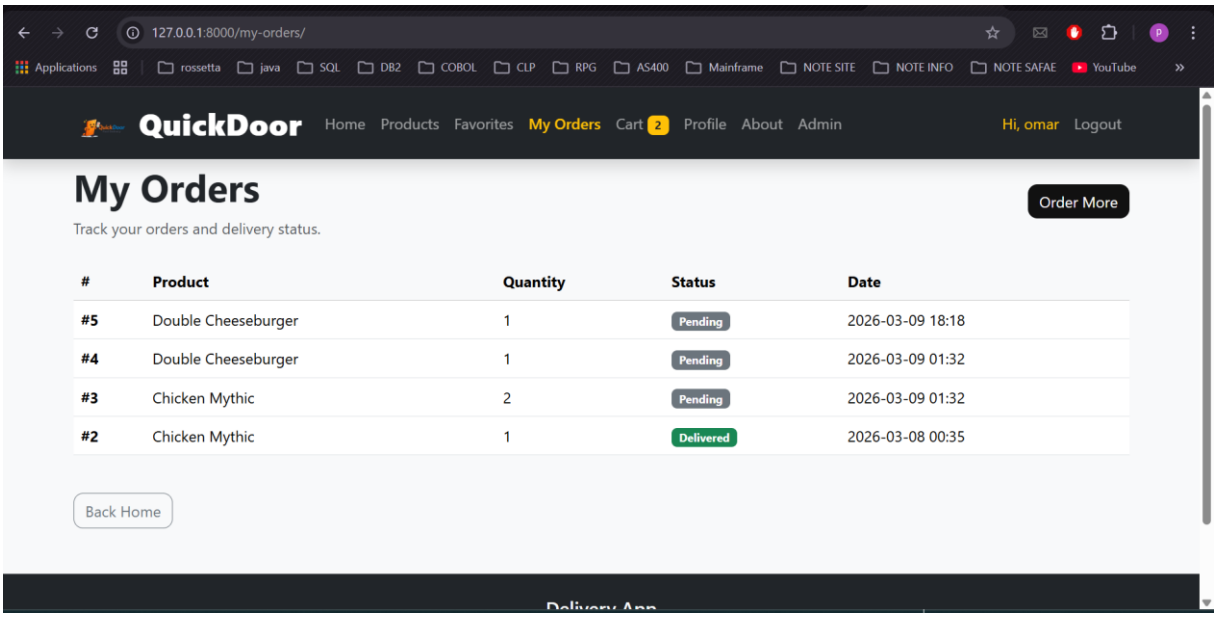


Figure 12 : Interface de la page des commandes

Cette page améliore l'expérience utilisateur en offrant une visibilité complète sur les commandes effectuées.

8 Conclusion et perspectives

Au terme de ce projet, nous avons conçu et développé une application web de livraison nommée **QuickDoor**, en utilisant le framework Django.

Ce projet nous a permis de mettre en pratique nos connaissances en développement web, notamment en conception de bases de données, en programmation backend avec Python/Django, ainsi qu'en développement frontend avec HTML, CSS et Bootstrap.

Nous avons également acquis des compétences en modélisation UML, en organisation du code et en gestion des fonctionnalités d'une application complète.

L'application développée offre plusieurs fonctionnalités telles que la gestion des produits, des commandes, des paniers ainsi que le suivi des livraisons, répondant ainsi aux besoins des utilisateurs.

Cependant, ce projet peut être amélioré en ajoutant de nouvelles fonctionnalités comme :

- L'intégration d'un système de paiement en ligne
- Le développement d'une application mobile
- L'amélioration du design et de l'expérience utilisateur
- L'ajout d'un système de géolocalisation pour le suivi des livraisons

En perspective, ces améliorations permettront de rendre l'application plus complète et plus performante.